

CS 5200: Introduction to Database Management Systems

Practicum 1: Analyzing Student SQL Errors Using Databases and AI

Team A

Ziming Yi | Ikenna Odukwe

Summer 2026 | Dr. Hazra Imran

1. Introduction

Understanding how students make errors when learning SQL is essential for designing effective educational interventions. Syntax and schema errors — queries that fail to execute due to incorrect syntax or references to undefined database objects — are among the most common obstacles for novice SQL learners. Identifying which SQL concepts generate the most errors, and what patterns those errors follow, can help instructors and adaptive learning systems provide targeted feedback.

This practicum investigates error patterns in the SQL-Engage dataset, a synthetic corpus of 1,500 annotated SQL queries developed at the Adaptive Learning and Gamification Lab (ALGL), Northeastern University Vancouver. Team A focuses on syntax and schema errors. Our core research question is: Which SQL concepts generate the most syntax and schema errors, and do certain error subtypes dominate specific concepts?

2. Dataset Description

2.1 SQL-Engage Dataset

The SQL-Engage dataset (v1.0) is a synthetic, emotion-aware SQL error corpus containing 1,500 annotated query instances. Each record captures six attributes:

- query: the SQL query text (PostgreSQL-style)
- error_type: one of syntax, schema, logic, or construction
- error_subtype: one of 23 specific error patterns
- emotion: simulated learner affective state (anger, calmness, happiness, neutral, sadness)
- feedback_target: a tailored feedback message for the error
- intended_learning_outcome: the conceptual objective of the query

Team A retained only syntax and schema error records, yielding 1,041 records (846 syntax, 195 schema).

2.2 Preprocessing and Data Cleaning

The following transformations were applied prior to loading into MySQL:

Step	Description	Records Affected
Filtering	Retained only syntax and schema error records; discarded logic (394) and construction (65) records	459 rows removed

Missing value check	All six columns checked for nulls; none found	0 rows affected
sql_concept derivation	A new column derived from query text by detecting SQL keywords (GROUP BY, HAVING, JOIN, ORDER BY, WHERE, DISTINCT, SELECT, INSERT, UPDATE, DELETE) in priority order	1,041 rows derived
Normalization	Five categorical columns extracted into lookup tables (ErrorType, ErrorSubtype, Emotion, SQLConcept, LearningOutcome) to satisfy 3NF	All rows
Text escaping	Single quotes escaped as " and backslashes as \\ for safe MySQL insertion	All rows

No records were dropped for data quality reasons. The sql_concept column was derived because the original dataset does not include a concept label; SQL keyword detection provides a reasonable proxy for the clause being practised.

3. Schema Design

3.1 Entity-Relationship Diagram

The ERD for the sql_engage_db database is shown in Figure 1 (see attached ERD.drawio.png). The schema consists of six tables: one central fact table (Submission) and five lookup tables (ErrorType, ErrorSubtype, Emotion, SQLConcept, LearningOutcome).

3.2 Tables and Normalization

The schema is in Third Normal Form (3NF). Each non-key attribute depends only on the primary key of its table, with no transitive dependencies. Enumerable values (error types, subtypes, emotions, SQL concepts) are stored in dedicated lookup tables and referenced via foreign keys.

Table	Primary Key	Foreign Keys	Description
ErrorType	error_type_id	—	Enumerates syntax and schema
ErrorSubtype	subtype_id	error_type_id	16 subtypes, each belonging to one ErrorType
Emotion	emotion_id	—	5 learner affective states
SQLConcept	concept_id	—	10 SQL keywords/clauses (derived)
LearningOutcome	outcome_id	—	130 unique learning objectives

Submission	submission_id	subtype_id, emotion_id, concept_id, outcome_id	Main fact table; 1 row per query instance
------------	---------------	---	---

The feedback_target column is stored directly in Submission because it is unique per record and cannot be normalized further without loss of meaning.

4. Error Pattern Analysis

4.1 Note on Dataset Limitations

The SQL-Engage dataset is a synthetic corpus without student identifiers or attempt sequences. Unlike a real student submission dataset, each row is an independent annotated instance. This means metrics such as 'average attempts before success' and 'error persistence rate' (defined as the proportion of students who repeat the same error) cannot be computed directly. We instead use frequency-based proxies: error concentration per concept approximates attempt difficulty, and subtype dominance approximates persistence. These limitations are acknowledged throughout.

4.2 Frequency of Each Error Subtype by SQL Concept (Required Query 1)

Table 3 shows the top error subtypes across key SQL concepts. WHERE is the most error-prone concept (469 total errors), followed by GROUP BY (153) and JOIN (152). Within WHERE, errors are spread across all 16 subtypes, suggesting broad difficulty. Within GROUP BY, aggregation misuse (37) and wrong positioning (22) dominate, indicating specific conceptual misunderstandings around aggregate functions.

SQL Concept	Top Error Subtype	Count	Error Type
WHERE	incorrect wildcard usage	58	syntax
GROUP BY	aggregation misuse	37	syntax
GROUP BY	wrong positioning	22	syntax
JOIN	non-standard operators	17	syntax
JOIN	undefined table	17	schema
SELECT	incorrect SELECT usage	27	syntax
SELECT	undefined function	27	schema
DISTINCT	incorrect DISTINCT usage	20	syntax
ORDER BY	wrong positioning	17	syntax

4.3 Proxy for Attempt Difficulty (Required Query 2)

We compute `avg_errors_per_subtype` (total errors divided by distinct subtypes observed) as a proxy for how concentrated errors are within a concept. A higher ratio means fewer error types dominate, suggesting learners repeatedly make the same mistakes.

SQL Concept	Total Errors	Distinct Subtypes	Avg Errors per Subtype
WHERE	469	16	29.31
SELECT	131	12	10.92
JOIN	152	15	10.13
GROUP BY	153	16	9.56
DISTINCT	33	6	5.50
ORDER BY	73	14	5.21
INSERT	22	8	2.75
SUBQUERY	5	3	1.67

WHERE has a substantially higher `avg_errors_per_subtype` (29.31) than all other concepts, driven primarily by incorrect wildcard usage (58 instances). This suggests WHERE clause errors are highly concentrated around a small number of recurring mistakes.

4.4 Error Persistence Proxy (Required Query 3)

For each SQL concept, we identify the dominant error subtype and compute its share of total errors for that concept. A dominance percentage above 40% is classified as High persistence.

SQL Concept	Dominant Subtype	Dominant Count	Total	Dominance %	Label
HAVING	misspelling	1	1	100.0%	High persistence
DISTINCT	incorrect DISTINCT usage	20	33	60.6%	High persistence
SUBQUERY	incorrect SELECT usage	3	5	60.0%	High persistence
GROUP BY	aggregation misuse	37	153	24.2%	Distributed
ORDER BY	wrong positioning	17	73	23.3%	Distributed
WHERE	incorrect wildcard usage	58	469	12.4%	Distributed

JOIN	non-standard operators / undefined table	17	152	11.2%	Distributed
------	--	----	-----	-------	-------------

DISTINCT and SUBQUERY show high persistence, meaning a single error subtype accounts for the majority of errors in those concepts. WHERE and JOIN, despite having the highest total error counts, have distributed error patterns, suggesting broader conceptual difficulty rather than a single recurring mistake.

4.5 Concept Difficulty Ranking (Required Query 4)

Concepts are ranked by total error count descending, then by subtype diversity ascending (fewer distinct error types at high volume indicates more focused, persistent difficulty).

Rank	SQL Concept	Total Errors	Subtype Diversity	Syntax	Schema
1	WHERE	469	16	401	68
2	GROUP BY	153	16	131	22
3	JOIN	152	15	113	39
4	SELECT	131	12	88	43
5	ORDER BY	73	14	61	12
6	DISTINCT	33	6	30	3
7	INSERT	22	8	15	7
8	SUBQUERY	5	3	4	1
9	UPDATE	2	2	2	0
10	HAVING	1	1	1	0

WHERE ranks first by a wide margin. The top four concepts (WHERE, GROUP BY, JOIN, SELECT) account for 905 of 1,041 errors (87%), suggesting that instructional effort should be concentrated on these fundamental clauses.

4.6 Additional Query 1: Emotion Distribution Across Error Subtypes

We examined the distribution of learner emotions across error subtypes to identify which errors are associated with the most negative affective states. The results show a roughly uniform emotion distribution (approximately 20% per emotion) across most subtypes, which is expected given the synthetic nature of the dataset. Notably, incorrect DISTINCT usage has the highest proportion of anger (24.6%), while incorrect SELECT usage is most associated with happiness (26.2%). While the synthetic data limits the

interpretive value of these findings, the analysis framework is directly applicable to real learner data.

4.7 Additional Query 2: Schema vs. Syntax Error Distribution by Concept

We computed the proportion of schema errors (referencing undefined objects) versus syntax errors (structural/grammatical) for each concept.

SQL Concept	Syntax Count	Schema Count	Total	Schema %
SELECT	88	43	131	32.8%
INSERT	15	7	22	31.8%
JOIN	113	39	152	25.7%
ORDER BY	61	12	73	16.4%
WHERE	401	68	469	14.5%
GROUP BY	131	22	153	14.4%
DISTINCT	30	3	33	9.1%

SELECT and INSERT have the highest schema error proportions (above 30%), suggesting that learners frequently reference undefined columns or functions in basic SELECT and INSERT statements. JOIN has a substantial schema error rate (25.7%), likely due to referencing incorrect table names. WHERE, despite being the most error-prone concept overall, is predominantly a syntax error concept (85.5% syntax), indicating that WHERE clause errors are mainly structural rather than object-reference failures.

5. LLM-Based Feedback Generation (Task 3)

5.1 Methodology

We used the Claude API (model: claude-sonnet-4-6) to generate targeted feedback for the five most frequent error patterns identified in Task 2. For each error pattern, a representative example query was retrieved from the database and passed to the model along with the error category, subtype, and intended learning outcome.

The following prompt template was used consistently for all five error patterns:

System: "You are a patient SQL tutor for students writing their first SQL queries. When shown a faulty query, you reply with ONE short feedback message of 2-4 sentences. The message must: (1) name what is wrong, (2) briefly explain why it is wrong, and (3)

tell the student what to do differently. Use an encouraging, beginner-appropriate tone. Do NOT rewrite the full corrected query for them and do NOT add headings, bullet points, or code blocks."

User: "A student practicing the SQL concept "{concept}" made an error. Error category: {error_type}. Error pattern: {subtype}. The student submitted this query: {query}. Correct expectation (what they should learn): {outcome}. Write the feedback message this student would receive."

5.2 Top 5 Error Patterns and Generated Feedback

Rank	Error Pattern	Type	Frequency	Generated Feedback (excerpt)
1	incomplete query	syntax	66	Great effort on writing a JOIN query — you're really close! The issue is in your WHERE clause: `r.Status` is an incomplete condition...
2	undefined column	schema	65	Great effort on structuring your ORDER BY query! The issue here is that one of the column names you've used doesn't match what's actually defined in the database schema...
3	undefined function	schema	65	Great effort on trying to combine string values together! The issue here is that `CONCAT_AS` is not a recognized SQL function...
4	undefined table	schema	65	Great effort on using the WHERE clause correctly! The issue here is that the table name "emp" doesn't exist in the database schema...
5	aggregation misuse	syntax	65	Great effort on using GROUP BY and COUNT together — you're on the right track! The issue is that you used COUNT(Log_ID) inside a WHERE clause...

All generated feedback was stored in the LLMFeedback table alongside the prompt text, model name, and frequency rank. The feedback consistently follows the required structure: identifying the error, explaining why it is wrong, and providing actionable guidance.

6. Structured Evaluation of LLM Feedback (Task 4)

6.1 Evaluation Rubric

Each team member independently evaluated all five feedback items on five dimensions: Accuracy, Specificity, Actionability, Clarity, and Appropriateness, each scored 1-4.

6.2 Evaluation Results

Error Pattern	Rater	Accuracy	Specificity	Actionability	Clarity	Appropriateness
incomplete query	Ziming Yi	3	4	4	4	4
incomplete query	Ikenna Odukwe	3	3	4	4	4
undefined column	Ziming Yi	4	3	4	4	4
undefined column	Ikenna Odukwe	4	3	4	3	4
undefined function	Ziming Yi	4	4	4	4	4
undefined function	Ikenna Odukwe	4	4	4	4	4
undefined table	Ziming Yi	4	4	4	4	4
undefined table	Ikenna Odukwe	4	4	3	4	4
aggregation misuse	Ziming Yi	4	4	4	4	4
aggregation misuse	Ikenna Odukwe	4	4	4	4	3

6.3 Inter-Rater Agreement

Inter-rater agreement was calculated as the percentage of dimension scores where the two raters differed by no more than 1 point. Out of 25 dimension comparisons, all 25 fell within a 1-point difference, yielding an inter-rater agreement of 100%. No dimension required re-scoring or discussion.

6.4 Average Scores per Dimension and Error Pattern

Error Pattern	Accuracy	Specificity	Actionability	Clarity	Appropriateness
incomplete query	3.00	3.50	4.00	4.00	4.00
undefined column	4.00	3.00	4.00	3.50	4.00
undefined function	4.00	4.00	4.00	4.00	4.00
undefined table	4.00	4.00	3.50	4.00	4.00
aggregation misuse	4.00	4.00	4.00	4.00	3.50

Overall, the LLM-generated feedback scored highly across all dimensions. The lowest scores were for Specificity on incomplete query (3.50) and undefined column (3.00), reflecting that these feedback messages were slightly more generic. The aggregation misuse feedback received the highest overall scores, likely because the error pattern involves a well-defined conceptual distinction (WHERE vs. HAVING) that the model could address precisely.

7. Discussion

Our analysis of 1,041 syntax and schema error records reveals several patterns with implications for SQL instruction:

First, WHERE clause errors dominate the dataset. With 469 errors and an avg_errors_per_subtype of 29.31, the WHERE clause is both the most common source of errors and the most concentrated. Incorrect wildcard usage (58 instances) is the single most frequent error in the dataset. This suggests that learners struggle specifically with pattern matching syntax, likely because the LIKE operator and wildcard characters (% , _) have no close analogue in everyday programming.

Second, GROUP BY and aggregation errors reflect a deep conceptual gap. Aggregation misuse (37 errors) and wrong positioning (22 errors) in GROUP BY queries suggest that learners do not fully understand when and how to use aggregate functions, particularly the distinction between WHERE and HAVING. These errors are consistent with known difficulties in SQL education.

Third, DISTINCT and SUBQUERY errors are highly persistent. Despite low absolute counts, these concepts show dominant single subtypes (60%+ concentration), meaning that when learners make errors on DISTINCT or SUBQUERY, they tend to make the same specific mistake. Targeted practice on these specific patterns could be high-value.

Fourth, SELECT and JOIN concepts have the highest schema error rates. This reflects a different kind of difficulty: learners know the structural syntax but are unsure of the actual column and table names in the schema. This suggests that schema familiarity exercises (rather than pure syntax drills) may be more effective for these concepts.

A key limitation of this analysis is that the SQL-Engage dataset is synthetic. Emotion labels and error distributions may not reflect real learner behavior. Additionally, the absence of student identifiers prevents session-level analysis of error persistence and progression.

8. Appendix: SQL Queries

Required Query 1: Error Frequency by SQL Concept

```
SELECT c.concept_name, es.subtype_name, et.type_name, COUNT(*) AS frequency FROM
Submission s JOIN ErrorSubtype es ON s.subtype_id = es.subtype_id JOIN ErrorType et ON
es.error_type_id = et.error_type_id JOIN SQLConcept c ON s.concept_id = c.concept_id
GROUP BY c.concept_name, es.subtype_name, et.type_name ORDER BY c.concept_name,
frequency DESC;
```

Required Query 2: Avg Errors per Subtype (Difficulty Proxy)

```
SELECT c.concept_name, COUNT(*) AS total_errors, COUNT(DISTINCT s.subtype_id) AS
distinct_subtypes, ROUND(COUNT(*) / COUNT(DISTINCT s.subtype_id), 2) AS
avg_errors_per_subtype FROM Submission s JOIN SQLConcept c ON s.concept_id =
c.concept_id GROUP BY c.concept_name ORDER BY avg_errors_per_subtype DESC;
```

Required Query 3: Dominant Subtype per Concept (Persistence Proxy)

```
WITH concept_subtype_counts AS (SELECT s.concept_id, s.subtype_id, COUNT(*) AS cnt
FROM Submission s GROUP BY s.concept_id, s.subtype_id), concept_totals AS (SELECT
concept_id, SUM(cnt) AS total_errors FROM concept_subtype_counts GROUP BY concept_id),
ranked AS (SELECT csc.concept_id, csc.subtype_id, csc.cnt, ct.total_errors, RANK()
OVER (PARTITION BY csc.concept_id ORDER BY csc.cnt DESC) AS rn FROM
concept_subtype_counts csc JOIN concept_totals ct ON ct.concept_id = csc.concept_id)
SELECT c.concept_name, es.subtype_name AS dominant_subtype, r.cnt AS dominant_count,
r.total_errors, ROUND(r.cnt * 100.0 / r.total_errors, 1) AS dominance_pct, CASE WHEN
r.cnt * 100.0 / r.total_errors > 40 THEN 'High persistence' ELSE 'Distributed' END AS
persistence_label FROM ranked r JOIN SQLConcept c ON c.concept_id = r.concept_id JOIN
ErrorSubtype es ON es.subtype_id = r.subtype_id WHERE r.rnk = 1 ORDER BY dominance_pct
DESC;
```

Required Query 4: Concept Difficulty Ranking (View)

```
CREATE OR REPLACE VIEW v_concept_difficulty AS SELECT c.concept_name, COUNT(*) AS
total_errors, COUNT(DISTINCT s.subtype_id) AS subtype_diversity, SUM(CASE WHEN
et.type_name = 'syntax' THEN 1 ELSE 0 END) AS syntax_errors, SUM(CASE WHEN
et.type_name = 'schema' THEN 1 ELSE 0 END) AS schema_errors, RANK() OVER (ORDER BY
COUNT(*) DESC, COUNT(DISTINCT s.subtype_id) ASC) AS difficulty_rank FROM Submission s
JOIN SQLConcept c ON s.concept_id = c.concept_id JOIN ErrorSubtype es ON s.subtype_id
= es.subtype_id JOIN ErrorType et ON es.error_type_id = et.error_type_id GROUP BY
c.concept_name; SELECT * FROM v_concept_difficulty ORDER BY difficulty_rank;
```

Additional Query 1: Emotion Distribution per Error Subtype

```
SELECT es.subtype_name, em.emotion_name, COUNT(*) AS count, ROUND(COUNT(*) * 100.0 /
SUM(COUNT(*) OVER (PARTITION BY es.subtype_name), 1) AS pct_within_subtype FROM
Submission s JOIN ErrorSubtype es ON s.subtype_id = es.subtype_id JOIN Emotion em ON
s.emotion_id = em.emotion_id GROUP BY es.subtype_name, em.emotion_name ORDER BY
es.subtype_name, count DESC;
```

Additional Query 2: Schema vs. Syntax Distribution by Concept

```
SELECT c.concept_name, SUM(CASE WHEN et.type_name = 'syntax' THEN 1 ELSE 0 END) AS
syntax_count, SUM(CASE WHEN et.type_name = 'schema' THEN 1 ELSE 0 END) AS
schema_count, COUNT(*) AS total, ROUND(SUM(CASE WHEN et.type_name = 'schema' THEN 1
ELSE 0 END) * 100.0 / COUNT(*), 1) AS schema_pct FROM Submission s JOIN SQLConcept c
ON s.concept_id = c.concept_id JOIN ErrorSubtype es ON s.subtype_id = es.subtype_id
JOIN ErrorType et ON es.error_type_id = et.error_type_id GROUP BY c.concept_name ORDER
BY schema_pct DESC;
```

Stored Procedure: GetTopErrorSubtypes(N)

```
CREATE PROCEDURE GetTopErrorSubtypes(IN top_n INT) BEGIN SELECT es.subtype_name,
et.type_name AS error_type, COUNT(*) AS frequency, RANK() OVER (ORDER BY COUNT(*)
DESC) AS rnk FROM Submission s JOIN ErrorSubtype es ON s.subtype_id = es.subtype_id
JOIN ErrorType et ON es.error_type_id = et.error_type_id GROUP BY es.subtype_name,
et.type_name ORDER BY frequency DESC LIMIT top_n; END;
```

Task 3: LLM Feedback Storage Query

```
SELECT lf.feedback_id, es.subtype_name, lf.generated_feedback FROM LLMFeedback lf JOIN
ErrorSubtype es ON lf.subtype_id = es.subtype_id ORDER BY lf.frequency_rank;
```

Task 4: Average Score per Dimension and Error Pattern

```
SELECT lf.feedback_id, es.subtype_name AS error_pattern, fe.dimension,
ROUND(AVG(fe.score), 2) AS avg_score FROM FeedbackEvaluation fe JOIN LLMFeedback lf ON
fe.feedback_id = lf.feedback_id JOIN ErrorSubtype es ON lf.subtype_id = es.subtype_id
GROUP BY lf.feedback_id, es.subtype_name, fe.dimension ORDER BY lf.feedback_id,
fe.dimension;
```

Task 4: Inter-Rater Agreement

```
SELECT z.feedback_id, es.subtype_name, z.dimension, z.score AS ziming_score, i.score
AS ikenna_score, ABS(z.score - i.score) AS difference, CASE WHEN ABS(z.score -
i.score) <= 1 THEN 'Agree' ELSE 'Disagree' END AS agreement FROM FeedbackEvaluation z
JOIN FeedbackEvaluation i ON z.feedback_id = i.feedback_id AND z.dimension =
i.dimension JOIN LLMFeedback lf ON z.feedback_id = lf.feedback_id JOIN ErrorSubtype es
ON lf.subtype_id = es.subtype_id WHERE z.rater_name = 'Ziming Yi' AND i.rater_name =
'Ikenna Odukwe' ORDER BY z.feedback_id, z.dimension;
```

AI Tool Usage Disclosure

In accordance with course policy, we disclose that Claude (Anthropic) was used during this practicum to assist with schema design review, SQL query debugging, and report drafting. All analytical decisions, interpretations, and final SQL queries were reviewed and validated by both team members.